

Observability — A Readable Companion to the OTP Docs

Chapter 9 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why the BEAM is unusually observable

Most runtimes are observed from the outside: logs, OS metrics, a profiler attached after the fact. The BEAM can be observed from the inside while it runs. You can attach a shell to a production node, list every process, read any process’s state, trace calls to one function in one process, and detach again, all without a restart.

That power cuts both ways. The same tools that let you inspect a live system can take it down if used carelessly: a trace on a hot function, a query that copies a million-message mailbox, a log line that formats a 100 MB state. This chapter is about seeing clearly without making things worse.

2. The building blocks

Tool	What it tells you	Where it ships
<code>logger</code>	Structured events from your code and from OTP (crashes, supervisor reports)	kernel
<code>sys</code>	State, status, statistics and event traces of one OTP process	stdlib
<code>dbg / trace</code>	Calls, returns, messages and scheduling of chosen processes and functions	runtime_tools / kernel
<code>erlang:memory/0</code> , <code>statistics/1</code> , <code>system_info/1</code>	Node-wide numbers: memory by type, run queues, atom and process counts	erts
<code>erlang:system_monitor/2</code>	Alerts for long GCs, long schedules, huge heaps, busy ports, long message queues	erts
<code>observer</code>	GUI over all of the above	observer
<code>recon</code> (library)	Safe production helpers: top-N processes, rate-limited tracing	external

Logger in brief

Events pass through primary filters, then per-handler filters, then a formatter. There are eight levels, from emergency to `debug`. The primary level defaults to `notice`. That means `info` events, including

supervisor progress reports, are hidden unless you lower it.

Two ways to log:

```
?LOG_ERROR("file missing: ~ts", [Name]),           %% format string
?LOG_ERROR("#{event => file_missing, file => Name, reason => enoent}) %% report
```

Prefer reports: they stay structured all the way to the handler, so they can be filtered, indexed and formatted as JSON without parsing strings. Prefer the `?LOG_*` macros over `logger:*` functions: they add location metadata and skip evaluating their arguments when the level is filtered out.

3. How the runtime actually does it

Logger protects itself by dropping

The standard handler watches its own queue. With the defaults, above 10 queued events callers are made to wait (sync mode); above 200 new events are dropped; above 1000 the queue is flushed. Separately, a burst limit allows 500 events per second per handler, and drops the rest until the window ends.

This keeps logging from killing the node, but it means that during an incident, exactly when events come fastest, some of them are thrown away.

sys talks to the process

`sys:get_state/1` and friends work by sending a *system message* to the target and waiting for a reply (default timeout 5000 ms). The process must be responsive to answer. A process that is stuck in a long callback, or has a huge mailbox, will time out. The docs are clear that these functions are “intended only to help with debugging.”

Tracing costs are paid by the traced process

When a traced function is called, the runtime builds a trace message and sends it to the tracer. Tracing a function called 100,000 times per second creates 100,000 messages per second into one tracer process, whose mailbox then grows without bound (chapter 2). Setting trace flags on `all` processes applies the cost everywhere. OTP 27 added trace sessions (`dbg:session_create/1`, the `trace` module) so separate tracing activities no longer interfere with each other.

4. Plausible but wrong

4.1 Paying to format logs nobody sees

```
handle_cast(Msg, State) ->
  logger:debug(io_lib:format("state: ~p", [State])),
  ...
```

`io_lib:format` runs on every call, before `logger` checks the level, even though debug is filtered out in production. With a large state this is the most expensive line in the server. Fix: `?LOG_DEBUG("state: ~p", [State])`, which skips everything when debug is off. For big terms, log selected fields in a report instead of the whole state.

4.2 Logging entire payloads

```
?LOG_ERROR("request failed: ~p", [Request]).    %% Request holds a 20 MB body
```

Formatting a 20 MB term takes real time and memory, then floods the handler into drop mode, and the next few hundred useful events are lost. It can also put personal data in logs. Fix: log identifiers and sizes (`#{request_id => Id, body_bytes => byte_size(Body)}`), and set `depth` and `chars_limit` in the formatter config as a safety net.

4.3 Tracing a hot function on every process

```
dbg:tracer(),
dbg:p(all, c),
dbg:tpl(json, decode, x).
```

If `json:decode` runs thousands of times per second, the tracer's mailbox explodes, the shell becomes unusable, and memory climbs until the node struggles. People have taken down production this way. Fix: trace one process (`dbg:p(Pid, c)`), or use a rate-limited tool such as `recon_trace:calls({json, decode, '_'}, 10)`, which stops by itself after 10 calls. Always end with `dbg:stop()`.

4.4 Reading a mailbox to count it

```
{messages, Msgs} = process_info(Pid, messages),
length(Msgs).
```

This copies every message into your process just to count them. On the process you are investigating, which has a million messages because it is in trouble, that copy can take down the shell or the node. Fix: `process_info(Pid, message_queue_len)`. To find the worst offenders, use `recon:proc_count(message_queue_len, 10)`.

4.5 `sys:get_state` as an API

```
current_config() ->
  #{config := C} = sys:get_state(config_server),
  C.
```

This bypasses the server's interface, depends on its internal state shape, and times out whenever the server is busy. It works in every test and fails in the first overloaded minute of production. Fix: add a real `get_config` call to the server, or publish the value where readers can get it directly (ETS or `persistent_term`, chapter 4).

4.6 Trusting the absence of a log line

```
%% post-incident review
%% "no 'payment_failed' errors in the logs, so no payments failed"
```

During the spike the handler was in drop mode or over its burst limit, so the missing errors may simply have been dropped. Fix: count important events with metrics (a counter per event, as with `telemetry` in Elixir), which do not drop under load, and alert on logger drop counts themselves.

4.7 Watching only OS-level metrics

```
CPU 40%, RSS 3.1 GB, all green
```

The BEAM hides its most useful failure signals from the OS: a growing message queue, run queue length, binary memory (chapter 3), atom count approaching the limit, or long GC pauses. Fix: export `erlang:memory()` by type, `erlang:statistics(total_run_queue_lengths)`, `system_info(atom_count)` and process counts. Install an `erlang:system_monitor/2` handler for `long_gc`, `long_schedule`, `large_heap` and long message queues, so pathologies announce themselves.

5. Review checklist and sources

- ☐ Code logs with `?LOG_*` macros, preferably as structured reports
- ☐ No log line formats whole states, payloads or unbounded terms
- ☐ Formatter `depth` / `chars_limit` are set; logger drop counts are monitored
- ☐ Important events are also counted as metrics
- ☐ Tracing in production is limited to specific processes or is rate-limited, and always stopped
- ☐ Mailbox sizes are read with `message_queue_len`, never `messages`
- ☐ `sys` functions are used only for debugging, never as an API
- ☐ BEAM-level metrics are exported and `system_monitor` is installed

Sources

- Logging, Kernel User's Guide: https://www.erlang.org/doc/apps/kernel/logger_chapter.html
- `sys` module reference: <https://www.erlang.org/doc/apps/stdlib/sys.html>
- `dbg` module reference: https://www.erlang.org/doc/apps/runtime_tools/dbg.html
- `erlang` module reference (system_monitor, memory, statistics):
<https://www.erlang.org/doc/apps/erts/erlang.html>
- recon library: <https://ferd.github.io/recon/>